Домашнее задание №2. Генерация кода

Курс NLP, НИУ ВШЭ

ФИО: Нефедов Дмитрий **Группа:** БПМИ-233

Дата: 25 октября 2026 года

1. Постановка задачи

Нужно по тексту математической задачи сгенерировать Python-код, который вычисляет ответ. По условиям задания требовалось обучить encoder-decoder Transformer с нуля, без дополнительных данных и предобученных моделей.

Качество оценивается по **pass@1**: для каждой задачи проверяется один сгенерированный вариант решения. Помимо качества, нужно выполнить ограничение по скорости: получить ответы на все 1 919 тестовых условий не более чем за **180 секунд на одной NVIDIA Tesla T4**.

2. Подготовка данных

Я использовал `SlavaYus/tinysms_qa_12m`. Из 11 838 003 строк официальной обучающей части было выделено **11 244 489 примеров для обучения** и **593 514 для validation**. Официальный test содержит **1 919 условий**.

Текст условия и код преобразовывались в последовательности токенов с помощью **byte-level BPE со словарём 8 192 токена**. Токенизация нужна, чтобы представить строки в виде числовых идентификаторов, с которыми работает модель. Токенизатор обучался только на train; validation и test в его обучение не входили.

Подготовленные последовательности сохранялись частями в формате **Arrow IPC**. Данные читались потоково, чтобы не держать миллионы текстов и токенизированных программ в оперативной памяти.

3. Первая модель E1

Сначала я обучил небольшой Transformer: **4 слоя энкодера и 4 слоя декодера**, размерность скрытых представлений **384** и **6 голов внимания**, FFN **1 536** с GELU и dropout **0,1**. В модели **19 693 056 параметров**, то есть около 19,69 млн. Такая конфигурация позволила сначала проверить весь путь от данных до готовой посылки.

Обучение заняло примерно одну эпоху; итоговая контрольная точка соответствует шагу **57 502**. На public модель получила **62,40% (599/960)**. Это показало, что даже небольшая модель уже умеет выдавать работоспособный код, но правильный математический ответ получается далеко не всегда. Следующим шагом я решил проверить более ёмкую модель.

4. Эксперименты и переход к V2

Перед полным обучением V2 я последовательно проверил данные, обучение и генерацию. Эти проверки решали разные задачи, поэтому их результаты нельзя напрямую сравнивать как качество моделей.

Проверка данных и реализации. Сначала чтение Parquet, разбиение и токенизация проверялись на 100 исходных примерах официального набора, затем была выполнена подготовка полного датасета. Проверялись исключение validation из обучения BPE, сохранение строк при кодировании и декодировании, чтение Arrow и восстановление повреждённого кэша. Перед V2 также сравнивались обычное и кэшированное вычисления в BF16; в сохранённой проверке максимальное расхождение равно нулю.

Пилотные этапы E1. Обучение запускалось с продолжениями на шагах 500, 2 000 и 10 000. После первых 500 шагов бюджет батча увеличен с 8 192 до 32 768 токенов. Validation token loss снизился с 3,385704 на шаге 500 до 0,368888 на шаге 56 000. Это помогало следить за обучением, но сам по себе низкий token loss не означает правильного решения задачи.

Переобучение на 128 примерах. Отдельную отладочную модель обучили на 128 исходных train-примерах. К шагу 500 она точно воспроизводила код для всех **128/128 примеров (100% exact code)**. Это проверка способности реализации выучить небольшой набор и корректно генерировать последовательности.

Короткие GPU-бенчмарки. Для `v2_small`, `v2_gelu` и `v2_main` проведены 30-шаговые замеры при нескольких бюджетах батча на RTX 5090. Ниже приведён один сопоставимый режим: логический бюджет 16 384 токена. Измерялись скорость вычислений и пиковая память, выделенная PyTorch.

Вариант	Параметры, млн	Target-токенов/с, тыс.	Память, GiB
`v2_small`	49,90	181,3	4,02
`v2_gelu`	100,76	142,3	5,83
`v2_main`	100,77	140,9	6,31

Меньшая модель оказалась быстрее. Два крупных варианта показали близкую скорость; `v2_main` требовала больше памяти в этом замере. Для полного обучения я выбрал `v2_main`.

5. Итоговая модель V2

V2 содержит **8 слоёв encoder и 4 слоя decoder**, `d_model=768`, **12 голов внимания** и **SwiGLU FFN размерности 2 048**. Использованы LayerNorm перед блоками, синусоидальное позиционное кодирование, общие embeddings encoder/decoder и общая с ними выходная матрица. Dropout равен **0,1**; всего **100 767 744 параметра**. Гипотеза состояла в том, что более широкий и глубокий encoder лучше обработает условие; decoder оставлен четырёхслойным, чтобы сдержать стоимость последовательной генерации.

Модель обучалась с нуля на **RTX 5090 в Vast.ai**, в режиме **BF16**, с оптимизатором **AdamW**. Максимальный learning rate — $3 \cdot 10^{-4}$: сначала warmup на 20 млн target-токенов, затем косинусное снижение до 0,1 от максимума. Планировщик зависит от числа обработанных target-токенов, поэтому учитывает различную длину батчей. Использовалась cross entropy по исходному коду с игнорированием PAD.

За **2 эпохи** выполнено **111 275 шагов**, обработано **22 488 978 предъявлений train-примеров** и **1 640 734 418 target-токенов**.

6. Результаты

В таблице разделены два вида оценки. **Public pass@1** получен через официальный Telegram-бот. **Confirm AST agreement** — внутренняя диагностика: ограниченный интерпретатор Python AST сравнивает числовые результаты с исходными эталонными программами. Из 2 088 confirm-примеров он поддерживает числовые эталоны для 1 578, поэтому знаменатель здесь именно 1 578.

Метрика	E1	V2	Изменение
Официальный public pass@1	62,40% 599/960	**74,27%** 713/960	+11,87 п.п.
Confirm AST agreement	68,82% 1086/1578	**76,30%** 1204/1578	+7,48 п.п.

Изменение public указано по опубликованным процентам: 74,27 – 62,40. По числу задач V2 решила на 114 больше.

На confirm V2 исправила 170 ошибок E1, но ошиблась в 52 случаях, которые E1 решала правильно. В сумме результат улучшился.

По public-отчёту V2 основной источник ошибок — **неверное вычисленное значение (25,31%)**. Ошибки выполнения составили около **0,42%**; отсутствующего кода и синтаксических ошибок на public не было. То есть основное ограничение осталось в переводе условия в правильные вычисления. Эти данные относятся к 960 public-задачам; подтверждённого private score в материалах нет.

7. Скорость генерации

Для T4 использованы **FP16**, **greedy decoding**, **batch size 32** и ограничение **384 новых токена**.

Полный проход по 1 919 задачам	Запуск 1, с	Запуск 2, с	Запуск 3, с
Исходный T4-бенчмарк	26,3019	26,3105	26,3202
Повторный Colab из GitHub	25,8330	25,5884	25,6908

8. Использование AI-агентов

В работе я использовал AI-агентов для помощи с кодом, оформлением репозитория и LaTeX.

Существенная часть помощи относилась к реализации: написание и доработка скриптов загрузки и токенизации данных, обучения Transformer, генерации на T4. Агенты также помогали готовить тесты, проверять кэширование. При финальной упаковке агенты помогали сравнивать E1 и V2, готовить воспроизводимые ноутбуки и проверять проект.

Реальные запуски я организовывал и выполнял в **Vast.ai и Google Colab**: обучение шло на RTX 5090, а скорость генерации проверялась на Tesla T4. Официальное качество проверялось через **Telegram-бот курса**. AI-агенты помогли быстрее подготовить реализацию, найти ошибки и оформить проверяемые результаты.

9. Заключение

В результате public pass@1 вырос с **62,40% до 74,27%**. Основной проблемой остались ошибки в математических вычислениях, а не в синтаксисе кода. Генерация всех 1 919 ответов на T4 занимает около 26 секунд, то есть укладывается в ограничение по времени.

Код и воспроизведение

Репозиторий: https://github.com/nefedovdima/hse-nlp-hw2

Веса и полный архив: https://github.com/nefedovdima/hse-nlp-hw2/releases/tag/v1.0.0

Для повторения инференса нужно открыть `notebooks/inference_colab_t4.ipynb` в Google Colab, выбрать T4, загрузить `HW2_CODE_FINAL.zip` из релиза и выполнить ячейки по порядку. На выходе получается `submission.jsonl`.